

House Split: GitOps delivery to a local Kubernetes cluster

DD2482 Automated Software Testing and DevOps, project report

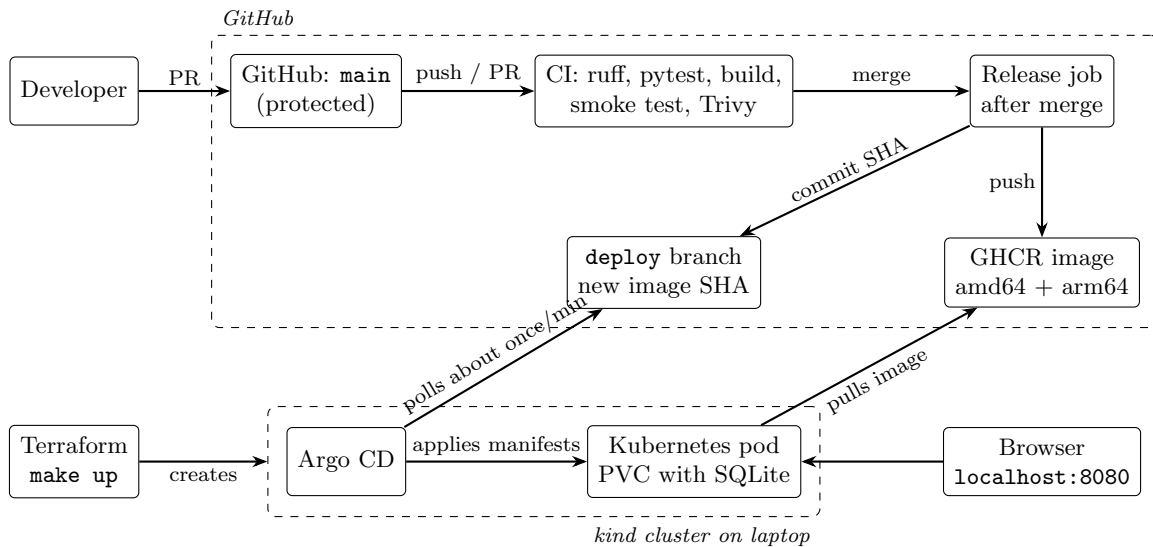
Tomás Brito (tmladb@kth.se) and Henrique Cavaco (hjcr@kth.se)

Repository: <https://github.com/tomasmbrito/house-split>

1 Goal

House Split is a small Flask app with SQLite storage for shared expenses between housemates. A user can add housemates, record who paid for an expense and choose who shares it. The app then shows everyone's balance and suggests payments to settle the total. Amounts are stored as integer cents, and leftover cents from a split are assigned one at a time so the shares still add up exactly. The settlement code repeatedly matches the person who owes the most with the person who is owed the most. This produces at most $n - 1$ payments, though it does not always find the absolute smallest number. We chose a simple app so we could focus on the DevOps work around it: checking changes automatically, building and publishing a container, and deploying it to a Kubernetes cluster that runs on a laptop. We wanted the setup to be reproducible without a cloud account.

2 Architecture and processes



The GitHub repository is our development platform. It holds the app and tests (`app/` and `tests/`), the Dockerfile, the Actions workflow, the Kubernetes manifests in `k8s/`, and Terraform configuration in `infra/`. Changes go through pull requests to the protected `main` branch. The rules require the CI checks to pass and the other team member to approve before a change is merged. This gives us one place to review changes and see whether they passed the same checks.

The workflow has three jobs. The `test` job installs the development dependencies, runs Ruff's lint and format checks, and runs 19 pytest tests. These include checks for amount parsing, expense sharing, balance calculation and the suggested settlement payments. The `docker` job runs after the tests; it builds the image, starts the container and calls its `/health` endpoint. It then runs Trivy and fails on critical vulnerabilities that have a fix. After a push to `main`, the `release` job runs only if the earlier jobs passed. It builds and publishes amd64 and arm64 images to GitHub Container Registry (GHCR), tagged with the commit SHA, and updates `k8s/kustomization.yaml` on the `deploy` branch with that tag.

Argo CD runs in the local cluster and watches the `deploy` branch. About once a minute it checks for changes, renders the manifests with kustomize and applies them to Kubernetes. The Deployment replaces the old pod, and the new pod pulls the tagged image from GHCR. The app's SQLite file is mounted from a PersistentVolumeClaim (PVC), so replacing the pod does not remove the stored data. In our runs, a change was live around two to four minutes after merge: roughly the CI time plus the wait for Argo CD's next check.

The environment itself is infrastructure as code. The `Makefile` provides `make up`, which runs Terraform

to create a one-node kind cluster, installs Argo CD from a pinned Helm chart and registers the app as an Argo CD Application. The kind node maps port 30080 to `localhost:8080`, where we can use the app in a browser. The Application watches `deploy` and `k8s/`; automated sync, pruning and self-healing are enabled. Pruning removes resources that are removed from Git, while self-healing reverts manual cluster changes to match Git. `make status` shows the application and Kubernetes resources, `make argocd` opens the Argo CD interface, and `make down` destroys the cluster. On our laptop, setup takes under two minutes, with the app's first sync about a minute later.

The access pattern is deliberately simple: CI can write to GHCR and the repository, but it cannot access the cluster. Argo CD reads the public repository, and the Kubernetes node pulls the public image. No cluster credentials have to be given to GitHub Actions. Dependabot checks the Python packages, base image, GitHub Actions and Terraform providers weekly; its update pull requests pass through the same checks. We also enabled GitHub secret scanning and push protection.

3 Design decisions

Pull-based delivery. The cluster runs on a laptop without a public address, so a GitHub runner cannot connect to it and run `kubectl apply`. Instead, Argo CD initiates the connection by polling GitHub. This suits the setup and means CI does not need cluster credentials. Git is also the record of the desired state: reverting a commit gives us a way to roll back, and Argo CD corrects manual changes. We chose Argo CD over Flux mainly because its UI makes sync status and rollouts easy to see and demonstrate.

kind and Terraform. We wanted a free environment that we and the graders could recreate locally. kind runs Kubernetes inside Docker. Terraform describes the cluster and the Argo CD installation, with provider and chart versions pinned. A shell script could create the same resources, but Terraform tracks state and makes the setup and teardown repeatable through the Makefile.

A separate deploy branch and SHA tags. CI updates the image tag on `deploy`, leaving `main` protected and without bot commits. The workflow pushes with `GITHUB_TOKEN`; those pushes do not trigger another workflow run, so they do not create a loop. The commit SHA identifies the code in the image and gives Argo CD a changed tag to detect. We build for arm64 as well as amd64 because kind on our Apple Silicon Macs uses arm64 nodes.

One replica and a pinned scanner. SQLite is stored as one file on a `ReadWriteOnce` volume. We therefore run one replica and use Kubernetes' `Recreate` strategy, which stops the old pod before starting the replacement. This avoids two pods using the same file at once, but creates a short outage during deployment. The Trivy action is pinned to a commit SHA because action tags can move; Dependabot can still propose a newer version.

4 Limitations and trade-offs

The project shows an automated delivery process, but it is not a production deployment. The cluster has one node and runs on a laptop, so it is unavailable when the laptop is off and does not demonstrate high availability, network-failure handling or scaling. The use of SQLite also ties the app to one replica and a short outage during each replacement. For a production service we would use a database service and a deployment strategy that supports multiple replicas.

Polling keeps the cluster private but can add up to about two minutes before a new version syncs (the 60-second interval plus random jitter). A webhook could shorten that wait, but GitHub would need a way to reach the cluster. There is also a small gap between scanning and publishing: Trivy scans the amd64 image in the Docker job, while the release job builds both architectures again. They use the same commit and Dockerfile, but the base image could change between builds. Building once and promoting the scanned image would remove this gap.

Every merge currently triggers a release, even when only documentation changes. Path filters could avoid that, but they complicate required status checks, so we kept one straightforward workflow. The release job force-pushes the `deploy` branch, which means that branch does not keep a full deployment history; Argo CD and GitHub Actions provide records of syncs and runs instead. Rollback is a revert on `main`, so it goes through CI and takes a few minutes rather than happening immediately.

The repository and GHCR package are public, which lets the cluster pull them without credentials. A

private repository would need image pull secrets. Terraform state is kept locally, which is manageable for one laptop but not ideal for a shared environment. The app has no secrets, so secret management is not covered. Finally, our tests cover app behavior through Flask’s test client and the container’s health endpoint, but we do not have an end-to-end test against the running Kubernetes deployment.

5 Use of AI

We used Claude as a tool during the project, particularly when building the first version of the app and getting the setup started. It suggested early versions of parts of the Flask app and its tests, as well as drafts of some project configuration, including the Dockerfile, CI workflow, Kubernetes manifests, Terraform files and Makefile. We also used it to explain tools and options we were less familiar with and to help investigate errors.

We made the main design choices, including the choice of app and the pull-based GitOps approach. We reviewed the generated material, changed parts that did not fit our setup, and decided what to keep before committing it. We configured the GitHub protections and security features, connected the components, and ran the pipeline and deployment on our own laptops. We also checked that changes reached the app and that its data survived a pod replacement. When problems came up, we investigated and fixed them, using Claude’s explanations as one source of help. Claude helped us get started, but we were responsible for the final design, integration and verification.

6 Where each criterion is covered

Criterion	In our project
Automated build and testing	<code>test</code> and <code>docker</code> jobs in <code>.github/workflows/ci.yml</code> (Section 2)
Automated deployment	<code>release</code> job (GHCR + <code>deploy</code> branch) and Argo CD sync (Sections 2, 3)
Infrastructure configuration	Terraform in <code>infra/</code> (kind, Argo CD, Application) and manifests in <code>k8s/</code>
Development platform	GitHub: pull requests, branch ruleset, Actions, GHCR
Quality or security automation	Ruff, Trivy, Dependabot, secret scanning and push protection
Use of AI-assisted tools	Section 5 and the README
Project repository	the repository above; the whole system runs with <code>make up</code> (see README)